

Ministère de l'Enseignement Supérieur et la Recherche Scientifique

Faculté des Sciences Exactes et Informatique



Département d'Informatique

Tp Module:

Bases De Données – BDD



© Mlle Guetarni.H

Insertion de données:

L'instruction **INSERT INTO** est utilisée pour insérer de nouveaux enregistrements dans une table. Il est possible de rédiger la INSERT INTO déclaration de deux manières :

- Spécifiez à la fois les noms de colonnes et les valeurs à insérer :

```
INSERT INTO table_name (column1, column2, column3, ...) VALUES (value1, value2, value3, ...);
```

- Si vous ajoutez des valeurs pour toutes les colonnes de la table, vous n'avez pas besoin de spécifier les noms de colonnes dans la requête SQL. Cependant, assurez-vous que l'ordre des valeurs est le même que celui des colonnes du tableau. Ici, la **INSERT INTO** syntaxe serait la suivante :

```
INSERT INTO table_name VALUES (value1, value2, value3, ...);
```

En SQL, pour effectuer une insertion multiple de données dans une table, vous pouvez utiliser la syntaxe suivante :



```
INSERT INTO nom_table (colonne1, colonne2, colonne3, ...) VALUES  
    (valeur1_colonne1, valeur1_colonne2, valeur1_colonne3, ...),  
    (valeur2_colonne1, valeur2_colonne2, valeur2_colonne3, ...),  
    (valeur3_colonne1, valeur3_colonne2, valeur3_colonne3, ...),  
    ...  
    (valeurN_colonne1, valeurN_colonne2, valeurN_colonne3, ...);
```

Pour récupérer toutes les colonnes (les champs ou les attributs) de tous les enregistrements (les lignes ou les tuples) présents dans une table spécifiée:

```
SELECT * FROM nom_table;
```

Le caractère "*" (asterisque) est un wildcard qui signifie "toutes les colonnes". Ainsi, cette requête retournera tout ce qu'il y a dans votre table "nom_table", sans avoir besoin de préciser explicitement chaque colonne.

Voici quelques types de données populaires :

INTEGER ou **INT**: pour les nombres entiers

DECIMAL ou **NUMERIC**: pour les nombres décimaux

REAL: pour les nombres réels

DOUBLE PRECISION: pour les nombres réels avec une précision supérieure

SMALLINT: pour les nombres entiers de petite taille

BIGINT: pour les nombres entiers de grande taille

BIT: pour les valeurs booléennes (true/false)

DATETIME: pour les dates et heures

TIMESTAMP: pour les dates et heures avec une précision finissante

CHAR: pour les chaînes de caractères de longueur fixe

VARCHAR: pour les chaînes de caractères de longueur variable

TEXT: pour les chaînes de caractères de longueur illimitée

BINARY: pour les données binaires de longueur fixe

VARBINARY: pour les données binaires de longueur variable

Utilisation de fichiers externes:



En SQL, il est possible d'exécuter des commandes SQL à partir d'un fichier texte en utilisant la commande **SOURCE** ou `\.`. Cette méthode est utile lorsque vous avez une longue requête ou plusieurs requêtes à exécuter, car elle vous permet de stocker les commandes SQL dans un fichier texte et de les exécuter en une seule fois.

Pour exécuter des commandes SQL à partir d'un fichier texte en utilisant la commande **SOURCE**, vous devez suivre les étapes suivantes :

- Ouvrez la console MySQL.
- Placez-vous dans le répertoire où se trouve le fichier texte contenant les commandes SQL.
- Tapez la commande **SOURCE** `nom_fichier.sql`; où `nom_fichier.sql` est le nom du fichier texte contenant les commandes SQL.

La commande **SOURCE** exécute toutes les commandes SQL contenues dans le fichier texte, une par une, dans l'ordre où elles apparaissent dans le fichier.

Alternativement, vous pouvez utiliser la commande `\.` pour exécuter des commandes SQL à partir d'un fichier texte. La syntaxe est la suivante :

`\. nom_fichier.sql`

Cette commande exécute également toutes les commandes SQL contenues dans le fichier texte, une par une, dans l'ordre où elles apparaissent dans le fichier.

En utilisant cette méthode, vous pouvez facilement exécuter des commandes SQL à partir d'un fichier texte, ce qui peut vous faire gagner du temps et vous permettre de travailler plus efficacement.

Insérer les données à partir d'un fichier formaté:



Pour insérer des données provenant d'un fichier formaté (comme un fichier CSV) directement dans une table MySQL, vous pouvez utiliser la commande **LOAD DATA INFILE**. Voici comment utiliser cette commande pour insérer des données provenant d'un fichier CSV :

Créez une table vide dans laquelle vous souhaitez insérer les données du fichier CSV. Assurez-vous que le fichier CSV contient les mêmes types de données que celles prévues dans la table MySQL.

Utilisez la commande **LOAD DATA INFILE** avec la syntaxe suivante :

```
LOAD DATA LOCAL INFILE person.csv'  
INTO TABLE Persons  
FIELDS TERMINATED BY ';' ENCLOSED BY ""  
LINES TERMINATED BY '\n' -- ou '\r\n' selon l'ordinateur et le programme utilisés pour  
créer le fichier  
(PersonID, LastName, FirstName, Address, City);
```


Voici ce que chaque partie de cette commande signifie :

- **LOAD DATA LOCAL INFILE 'person.csv':** Cette partie indique à MySQL de charger les données à partir du fichier 'person.csv'. L'option LOCAL spécifie que le fichier est situé localement sur le client MySQL.
- **INTO TABLE Persons:** Cela spécifie que les données doivent être chargées dans la table nommée 'Persons'.
- **FIELDS TERMINATED BY ';' :** Cela indique que les colonnes dans le fichier CSV sont séparées par des points-virgules.
- **ENCLOSED BY '"':** Cela spécifie que les valeurs des colonnes sont encadrées par des guillemets doubles.
- **LINES TERMINATED BY '\n':** Cela indique que les lignes du fichier CSV sont terminées par un saut de ligne.

Sélection de données:



Dans le contexte de la sélection et de l'affichage de données en SQL, voici des exemples et des détails sur les points mentionnés :

Syntaxe de la requête **SELECT** :

La syntaxe de base de la requête **SELECT** est la suivante :

```
SELECT colonne1, colonne2, ... FROM nom_table;
```

Cette requête permet de sélectionner des données spécifiques depuis une table.

Sélection de données répondant à certaines conditions :

Pour sélectionner des données répondant à certaines conditions, vous pouvez utiliser la clause **WHERE** dans votre requête **SELECT** :

```
SELECT * FROM nom_table WHERE condition;
```

Par exemple :

```
SELECT * FROM Persons WHERE FirstName = 'Yassine';
```

Tri des données :

Pour trier les données selon une ou plusieurs colonnes, utilisez la clause **ORDER BY** :

```
SELECT * FROM nom_table ORDER BY colonne ASC|DESC;
```

Par exemple :

```
SELECT * FROM produits ORDER BY prix DESC;
```


Élimination des données en double :

Pour éliminer les doublons des résultats, utilisez **DISTINCT** dans votre requête **SELECT** :

```
SELECT DISTINCT colonne FROM nom_table;
```

Par exemple : `SELECT DISTINCT City FROM Persons;`

Récupération d'une partie des données :

Pour récupérer seulement une partie des données, vous pouvez utiliser **LIMIT** pour limiter le nombre de lignes retournées :

```
SELECT * FROM nom_table LIMIT 10;
```

Cela vous permettra de récupérer uniquement les 10 premières lignes. Vous pouvez également utiliser **OFFSET** pour spécifier à partir de quelle ligne commencer.

Remarque:

Pour illustrer l'utilisation de **OFFSET** en SQL pour spécifier à partir de quelle ligne commencer la récupération des données, voici un exemple :

```
SELECT * FROM Persons ORDER BY PersonID LIMIT 5, 10;
```

Dans cet exemple, la clause **LIMIT 5, 10** signifie que nous commençons à récupérer les données à partir de la 6ème ligne (**OFFSET 5**) et nous limitons le résultat à un total de 10 lignes. Cela signifie que nous récupérerons les lignes 6 à 15 de la table "Persons".

Sélection de données:



Les opérations de comparaison en SQL permettent de comparer des valeurs entre elles pour déterminer si elles sont égales, supérieures ou inférieures les unes aux autres. Voici quelques exemples et détails sur les opérations de comparaison en SQL:

Opérateurs de comparaison :

Les opérateurs de comparaison les plus couramment utilisés en SQL sont les suivants :

= : égal à.

<> ou **!=** : différent de.

< : inférieur à.

> : supérieur à.

<= : inférieur ou égal à.

>= : supérieur ou égal à.

Les opérateurs de comparaison sont généralement utilisés dans la clause **WHERE** d'une requête **SELECT** pour filtrer les résultats en fonction de certaines conditions. Par Par

exemple : **SELECT * FROM** clients **WHERE** age **>=** 18;

Cette requête sélectionne tous les clients dont l'âge est supérieur ou égal à 18 ans.

Opérateurs logiques :

Les opérateurs logiques permettent de combiner plusieurs conditions dans une requête **SELECT**. Les opérateurs logiques les plus couramment utilisés en SQL sont les suivants :

AND : retourne les résultats qui satisfont toutes les conditions spécifiées.

OR : retourne les résultats qui satisfont au moins l'une des conditions spécifiées.

NOT : retourne les résultats qui ne satisfont pas la condition spécifiée.

Les opérateurs logiques sont généralement utilisés pour combiner plusieurs conditions dans une clause **WHERE**.

Par exemple : **SELECT * FROM clients WHERE age >= 18 AND ville = 'Paris';**

Cette requête sélectionne tous les clients dont l'âge est supérieur ou égal à 18 ans et qui habitent à Paris.

Élargir les possibilités de la clause WHERE:



Bien que la clause **WHERE** en SQL permettra généralement de filtrer les résultats en fonction de conditions simples telles que l'égalité ou l'inégalité, elle offre également de nombreuses autres fonctionnalités qui augmentent significativement ses capacités. Voici quelques exemples et détails sur ces fonctionnalités :

Comparaison avec une chaîne de caractères incomplète :

La clause **WHERE** accepte des expressions régulières pour filtrer les résultats en fonction de parties spécifiques de chaînes de caractères. Par exemple, pour trouver toutes les personnes dont le nom commence par une certaine lettre, vous pouvez utiliser l'expression régulière **LIKE** :

```
SELECT * FROM Persons WHERE nom LIKE 'A%';
```

Ce code renvoie toutes les personnes dont le nom commence par la lettre 'A'.

Comparaison avec un intervalle de valeurs :

La clause **WHERE** accepte des expressions de comparaisons avec des intervalles de valeurs. Par exemple, pour trouver toutes les personnes dont l'âge se situe entre 2 et 5, vous pouvez utiliser l'opérateur **BETWEEN** :

```
SELECT * FROM animals WHERE age BETWEEN 2 AND 5;
```

Comparaison avec un ensemble de valeurs :

La clause **WHERE** accepte des expressions de comparaisons avec des ensembles de valeurs. Par exemple, pour trouver tous les animaux dont l'espèce se trouve dans un ensemble de trois espèces spécifiques ('Canard', 'Chatte', 'Chien'), vous pouvez utiliser l'opérateur **IN** :

```
SELECT * FROM animaux WHERE espèce IN ('Canard', 'Chatte', 'Chien');
```

Suppression de données:

La commande utilisée pour supprimer des données est **DELETE**. Cette opération est irréversible.

On utilise la clause **WHERE** de la même manière qu'avec la commande **SELECT** pour préciser quelles lignes doivent être supprimées:

```
DELETE FROM nom_table WHERE critères;
```

Par exemple: `DELETE FROM Persons WHERE FirstName = 'Yassine';`

Modification de données:

La modification des données se fait grâce à la commande **UPDATE**.

Par exemple: `UPDATE Persons SET FirstName='Yasser' WHERE id=21;`